



AI ASSISTENT PRO KVKLI

Maturitní práce

Autor	Jakub Havlas
Obor	Informační technologie
Vedoucí práce	Mgr. Michal Stehlík
Školní rok	2025/2026
Počet stran	43
Počet slov	7768

Anotace

Tato práce se zabývá návrhem a implementací AI asistenta pro webové stránky knihovny kvkli.cz. Cílem aplikace je zlepšit orientaci uživatelů na webu a umožnit sémantické vyhledávání v knihovním katalogu na základě přirozeného jazykového dotazu.

Součástí práce je také vývojový návrh infrastruktury pro provoz aplikace, včetně vytvoření testovacího prostředí, kontejnerizace pomocí technologie **Docker** a implementace **CI/CD** pipeline pro automatizované nasazování aplikace.

Aplikace je implementována jako webová služba využívající framework Next.js, rozhraní GraphQL a nástroj Prisma ORM pro práci s databází.

Summary

This thesis focuses on the design and implementation of an AI assistant for the kvkli.cz library website. The goal of the system is to improve user navigation on the website and enable semantic searching within the library catalog based on natural language queries.

The thesis also includes the design of the application infrastructure, including the creation of a testing environment, containerization using Docker technology, and the implementation of a CI/CD pipeline for automated application deployment.

The application is implemented as a web service using the Next.js framework, the GraphQL interface, and Prisma ORM for database interaction.

Čestné prohlášení

Prohlašuji, že jsem tuto práci vypracoval/a samostatně. V práci jsem použil/a pouze zdroje uvedené v seznamu literatury a všechny převzaté části, včetně textů, obrázků, kódu a výstupů umělé inteligence, jsou řádně označeny a citovány.

V Liberci dne 10.03.2026

.....

Jakub Havlas

Obsah

Úvod.....	1
1 Použité technologie (tech-stack).....	3
1.1 Role jazyka JavaScript	3
1.2 Next.js (2)	3
1.3 PostgreSQL + Prisma ORM (3)	3
1.4 Apollo + GraphQL (4) (5)	3
1.5 JEST – testování (6)	4
1.6 Docker Compose (7)	4
1.7 ChromaDB (8).....	4
1.8 CI/CD pipelines.....	4
1.9 OpenAI – jazykový model a embeddingy (9).....	5
2 Proces návrhu a implementace.....	6
2.1 AI API provider.....	6
2.1.1 Průzkum.....	6
2.2 OAI PMH (10) (11).....	6
2.2.1 Způsob vytěžování databáze knih	7
2.3 Kontext pro chatbota	8
2.3.1 Kontextové porozumění textu a embeddingy (12) (13) (14)	9
2.3.2 Vektorová databáze (7)	9
2.3.3 Normalizace CSV v jazyce Python.....	10
2.4 Function Calling	11
2.5 Next.js aplikace	12
2.5.1 Frontend (16)	13
2.6 Aplikační architektura.....	16
2.6.1 Datová vrstva (Prisma + PostgreSQL) (3)	16
2.6.2 API vrstva (GraphQL + Apollo) (4)	17
2.6.3 Aplikační logika (Resolver – Service Architektura).....	17
2.6.4 Integrace AI	18
2.6.5 Zpracovávání externích dat (18).....	19
3 Nasazení.....	21
3.1 VPS	21
3.1.1 Dockerfile a Docker Compose	21

3.1.2	CI/CD pipelines	22
3.2	Produkce.....	23
3.2.1	První testovací kolo	24
3.2.2	Fungování stránky kvkli.cz (19) (20) (21) (22)	24
3.2.3	Widget.....	24
3.2.4	Následné nasazení	25
3.2.5	Budoucnost nasazování	25
4	Bezpečnost.....	26
4.1	Bezpečnostní řešerše.....	26
4.1.1	Middleware + CORS (23)	26
4.1.2	Prompt limit cookies (24).....	26
4.1.3	Zabezpečení administrativního rozhraní	27
4.1.4	Heartbeat endpoint	27
5	Identifikované problémy a jejich řešení.....	28
5.1	Kontejnerizační a konfigurační problémy	28
5.2	Problémy integrace vektorové databáze	28
5.3	Multijazyčné prostředí projektu.....	29
5.4	Problematika RAG struktury	29
5.4.1	RAG struktura v katalogu	29
5.4.2	RAG struktura ve webové stránce	29
5.4.3	RefaktORIZACE A STABILIZACE SYSTÉMU	30
6	Budoucnost této práce	31
	Závěr	32
	Seznam zkratk a odborných výrazů	33
	Seznam obrázků	36
	Použité zdroje	37
A.	Seznam příložených souborů	I

Úvod

Rozvoj umělé inteligence v posledních letech zásadně proměňuje způsob, jakým uživatelé pracují s informacemi. Významnou roli v tomto trendu hrají zejména velké jazykové modely (**LLM**), které umožňují pokročilé porozumění přirozenému jazyku, generování odpovědí a sémantické vyhledávání nad rozsáhlými datovými soubory. Tyto technologie se postupně stávají běžnou součástí komerčních i veřejných digitálních služeb.

Tato práce tematicky navazuje na předchozí práci autora Adama Kaliny s názvem „Marketingový AI asistent“ jehož cílem bylo vytvořit **backendové** řešení pro inteligentního asistenta v oblasti online marketingu. Na rozdíl od tohoto projektu se však předkládaná práce zaměřuje na prostředí veřejné instituce, konkrétně na Krajskou vědeckou knihovnu v Liberci, a zkoumá možnosti využití umělé inteligence pro zlepšení přístupu uživatelů k informacím a knihovnímu fondu. (1)

Představení problému

Na základě konzultací s panem Miroslavem Filandou, IT specialistou a místním správcem aplikací, byla identifikována potřeba zjednodušit přístup uživatelů k informacím o službách knihovny. Přestože web kvkli.cz obsahuje všechny důležité údaje (otevírací dobu, registraci, poplatky či kulturní akce), jejich vyhledání není vždy rychlé a intuitivní.

Další problém se týká vyhledávání knih. Katalog je provozován na technologicky zastaralém řešení, které omezuje možnosti moderního dotazování a vykazuje funkční limity. Uživatelé často znají pouze téma, děj nebo postavy, nikoli konkrétní název či autora, zatímco tradiční katalogy jsou založeny na přesných metadatech. To zvyšuje nároky na personál a snižuje uživatelský komfort.

Součástí zadání bylo také vytvoření administračního rozhraní („backoffice“), které umožní zaměstnancům monitorovat provoz chatbota a analyzovat uživatelskou zpětnou vazbu.

Cílem je proto vytvořit inteligentního asistenta využívajícího sémantické vyhledávání, který umožní orientaci na webu, odpovídání na časté dotazy a hledání knih podle obsahového popisu. Řešení musí být bezpečné, škálovatelné a schopné pracovat s rozsáhlými a částečně nekonzistentními bibliografickými daty.

Přínos

Primárními uživateli budou návštěvníci webu KVCLI, kterým chatbot ušetří čas při hledání základních informací a umožní jim objevovat knižní tituly na základě vágních

popisů děje, což stávající katalog neumožňuje. Pro instituci samotnou projekt znamená modernizaci služeb a snížení zátěže pracovníků na informacích.

Cíle práce

Na základě konzultací s vedoucím práce a zadavatelem byl definován soubor funkčních a technických požadavků (viz příloha). Z těchto požadavků vycházejí následující cíle práce:

- Navrhnout a vybrat vhodný jazykový model pro roli knihovního asistenta.
- Navrhnout a implementovat **frontendovou** komponentu integrovatelnou do webových stránek kvkli.cz.
- Navrhnout a realizovat administrativní rozhraní (backoffice) pro správu systému a sběr uživatelské zpětné vazby.
- Zajistit nasazení aplikace do vývojového prostředí včetně základní konfigurace infrastruktury.

1 Použité technologie (tech-stack)

Tato kapitola popisuje výsledný „**tech-stack**“ zvolený pro realizaci aplikace. Výběr jednotlivých technologií byl podmíněn požadavky na škálovatelnost, dlouhodobou udržitelnost, efektivitu vývoje a možnost jednoduchého a reprodukovatelného nasazení do produkčního prostředí. Důraz byl kladen rovněž na kompatibilitu mezi jednotlivými komponentami systému a na dostupnost kvalitního vývojářského ekosystému.

1.1 Role jazyka JavaScript

JavaScript se stal dominantním jazykem moderního webu. Díky běhovému prostředí Node.js a nadstavbě TypeScript, která do jazyka přináší statickou typovou kontrolu, je možné vyvíjet robustní full-stack aplikace v rámci jednoho jazykového standardu. To výrazně urychluje vývoj a usnadňuje údržbu kódu.

1.2 Next.js (2)

Pro vývoj webové aplikace byl zvolen framework Next.js. Jedná se o moderní React framework umožňující vývoj frontendové i backendové části v rámci jednoho projektu.

Next.js nabízí podporu **server-side renderingu**, **API routes** a optimalizaci výkonu aplikace. V kontextu této práce bylo klíčové, že umožňuje provozovat GraphQL server, administrační rozhraní (backoffice) i chatovací **widget** v jedné aplikaci.

1.3 PostgreSQL + Prisma ORM (3)

Jako relační databáze byl zvolen PostgreSQL, který je robustním a osvědčeným databázovým systémem vhodným pro produkční nasazení. Slouží k ukládání konverzací, promptů a uživatelské zpětné vazby.

Pro práci s databází byl použit nástroj Prisma ORM. Ten umožňuje definovat datový model pomocí přehledného schématu a generuje typově bezpečného klienta pro Node.js. Výsledkem je konzistentní a bezpečná práce s daty bez nutnosti psát surové SQL dotazy.

1.4 Apollo + GraphQL (4) (5)

API vrstva byla realizována pomocí technologie GraphQL a knihovny Apollo. GraphQL umožňuje klientovi definovat přesnou strukturu požadovaných dat, čímž se minimalizuje přenos nadbytečných informací.

Apollo bylo využito jako klientská i serverová knihovna pro správu dotazů, mutací a cachování dat. Tato kombinace umožnila vytvořit flexibilní a efektivní komunikační rozhraní mezi frontendem a backendem.

1.5 JEST – testování (6)

Pro testování aplikační logiky byl použit nástroj Jest. Jedná se o populární testovací framework pro JavaScriptové aplikace.

Jest byl využit zejména pro tvorbu **jednotkových testů** servisní vrstvy, čímž bylo zajištěno, že funkce (např. práce s databází nebo generování odpovědí) fungují správně i po budoucích úpravách kódu. Dále byl implementován **smoke test** pro kontrolu běhu aplikace po nasazení.

1.6 Docker Compose (7)

Pro kontejnerizaci aplikace byl použit Docker a pro orchestraci jednotlivých služeb nástroj **Docker Compose**.

Aplikace je rozdělena do několika kontejnerů – Next.js aplikace, PostgreSQL databáze, ChromaDB a Nginx **reverzní proxy**. Docker Compose umožňuje jejich společné spuštění pomocí jednoho konfiguračního souboru, což výrazně zjednodušuje vývoj i produkční nasazení.

1.7 ChromaDB (8)

Pro ukládání **embeddingů** knih a realizaci sémantického vyhledávání byla zvolena vektorová databáze ChromaDB.

ChromaDB umožňuje ukládání vektorových reprezentací textu a efektivní vyhledávání na základě podobnosti. V kombinaci s embedding modelem tak tvoří základ pro implementaci principu **Retrieval-Augmented Generation** (dále jen RAG), který umožňuje chatbotovi doporučovat knihy podle jejich obsahu, nikoli pouze podle přesné textové shody.

1.8 CI/CD pipelines

Pro automatizaci testování a nasazování byly vytvořeny CI/CD pipelines pomocí GitHub Actions.

Pipeline se spouští automaticky při každém git push do hlavní větve repozitáře. Nejprve probíhá testovací fáze (instalace závislostí, spuštění testů) a následně, pokud je vše v pořádku, fáze nasazovací. Ta zajistí připojení na **VPS** server, synchronizaci kódu a restart Docker kontejnerů.

Tím je zajištěno, že do produkce je vždy nasazena otestovaná verze aplikace a celý proces je plně automatizovaný.

1.9 OpenAI – jazykový model a embeddingy (9)

Pro generování odpovědí chatbota bylo využito rozhraní OpenAI, konkrétně jazykový model GPT-4o mini. Tento model zajišťuje zpracování přirozeného jazyka, správu konverzační historie a podporu tzv. **function calling**, například pro vyhledávání v katalogu nebo získávání informací z webového prostředí knihovny.

Pro převod textových dat na vektorové reprezentace byl použit embedding model *text-embedding-3-small*. Výsledné embeddingy jsou ukládány do databáze ChromaDB a využívány při každém uživatelském dotazu pro sémantické vyhledávání relevantního obsahu.

V průběhu návrhu byly zvažovány i alternativní modely s vyšší kapacitou, avšak s ohledem na náklady, latenci odpovědi a charakter zamýšleného použití byl zvolen model, který představuje vhodný kompromis mezi výkonem a provozní efektivitou.

2 Proces návrhu a implementace

V této kapitole bude popsán vývoj aplikace, veškeré další možnosti, které byly při vývoji zváženy a zároveň bude vývoj zasezen do chronologického kontextu vývoje.

2.1 AI API provider

Vzhledem k finanční náročnosti trénování vlastních velkých jazykových modelů bylo rozhodnuto využít komerčně dostupné API třetích stran. Důležité pro výběr LLM API byly určeny tyto parametry:

1. Cena
2. Způsob implementace pro JavaScript
3. podpora RAG

2.1.1 Průzkum

První byl proveden průzkum o možnostech LLM API providerů. Na začátku byl prozkoumán ChatGPT od Open AI, Claude od Anthropic a Gemini od společnosti Alphabet. Z průzkumu vyšlo jasně nejlépe API od OpenAi, protože nabízela z daleka nejlepší ceny a měla kvalitní dokumentaci. V budoucnu byly ještě prozkoumány možnosti LLM **LLaMA** od společnosti Meta. U této cesty by bylo zajímavé to, že by se nainstaloval pouze lokální model, to by umožnilo zmenšit použité finanční prostředky na úplné minimum, ale zvýšilo by to využití výpočetní kapacity serveru.

2.2 OAI PMH (10) (11)

Po komunikaci s vývojářem firmy z Cosmotron s.r.o. nám bylo potvrzeno, že metadata knihovního katalogu jsou veřejně dostupná prostřednictvím OAI-PMH rozhraní. Dále vývojář dodal dokument pro práci s jejich harvest API. Jde o OAI-PMH provider, který cituji:

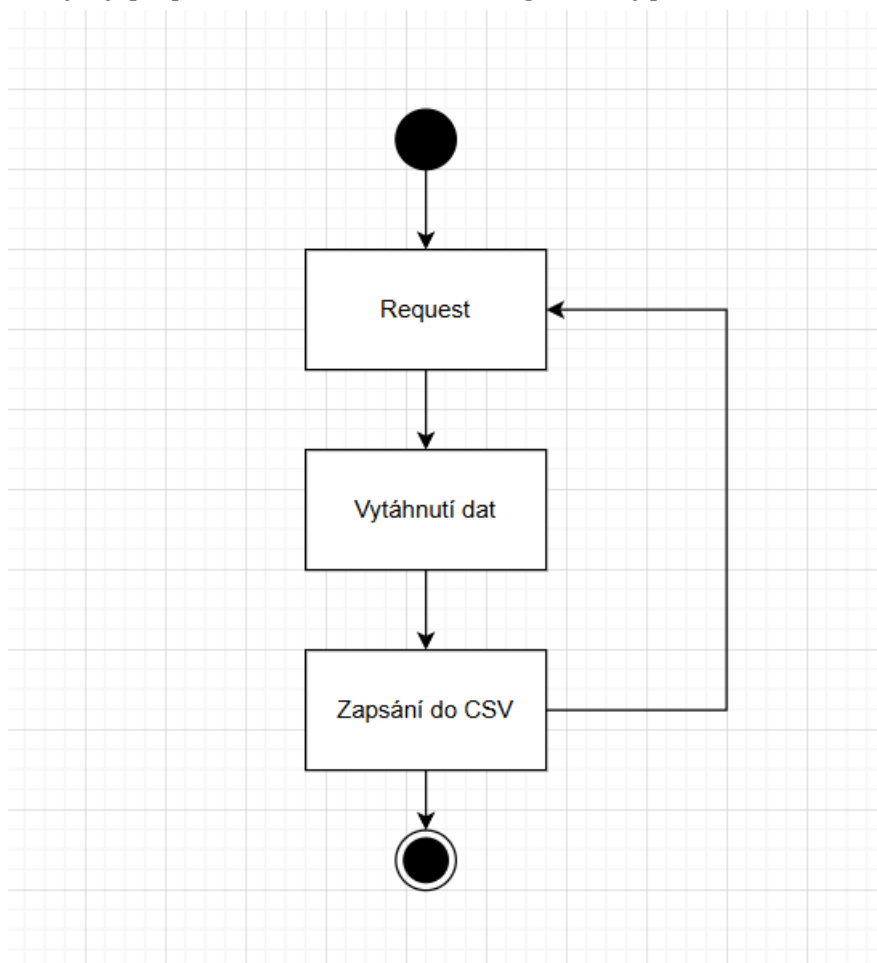
Komunikační modul OAI-PMH provider je určen k poskytování záznamů metadat vzdáleným harvesterům (sklízečům) ve standartních formátech XML Marc, Dublin Core apod (...)

Ve zkratce jde o dokument popisující harvest metodu, která funguje tak, že uživatel pošle první požadavek, kde si vyfiltruje, jaká data chce, z jaké kolekce a podobně. Získá první část souboru, na konci téhož souboru se bude nacházet „resumptionToken“, který musí z dat sebrat a poslat další požadavek pro získání dalšího části. Tímto způsobem lze tedy získat data.

2.2.1 Způsob vytěžování databáze knih

Pro implementaci vytěžování byl zvolen jazyk C#, a to z důvodu dostupnosti kvalitních nástrojů pro paralelní zpracování a silné typové kontroly. Celkem bylo nutné realizovat přibližně 9000 HTTP požadavků. Každý požadavek vrátí zhruba 25 KB xml:marc soubor, z něhož bylo potřeba vytáhnout data a následně je vložit do **CSV**.

První verze programu nefungovaly dostatečně na velikost databáze. Na začátku totiž nebylo vězeno, jak velká je databáze. První vytěžovací algoritmus byl zcela sekvenční, který by po převedení do use case diagramu vypadal takto:

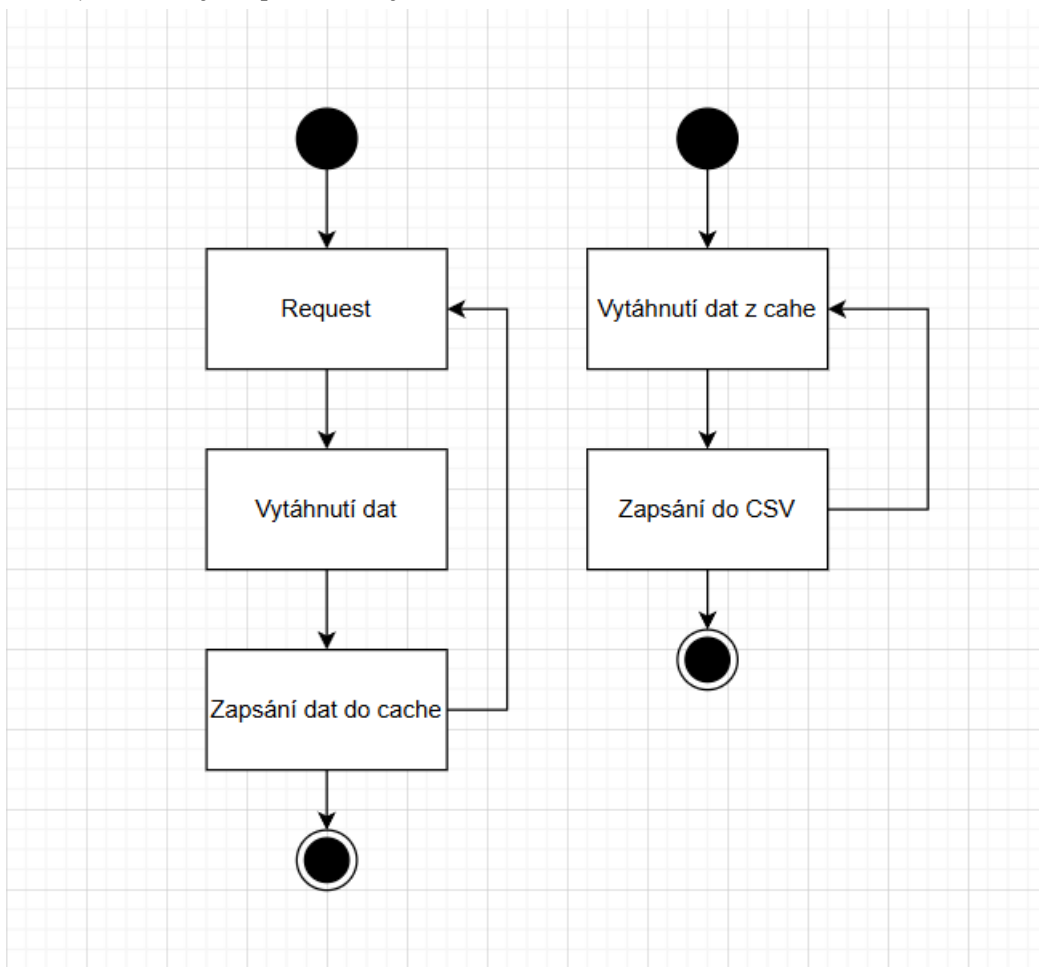


Obrázek 1 vývojový diagram vytěžování

Toto řešení se projevilo jako zcela neefektivní, protože po spuštění se ukázalo, že je časově nemožné, nevyužívá dostatečně systémových prostředků a celkový čas běhu byl v řádu několika desítek hodin. Bylo nutné optimalizovat výkon. První návrh řešení pro zrychlení je paralelizace. Tím vzniklo řešení číslo 2, které s malými změnami je používáno doposud.

V tomto řešení bylo třeba vyřešit synchronizaci mezi vlákny. Zatímco první vlákno plnilo paměťovou frontu (cache) surovými daty z API, druhé vlákno tato data asynchronně přepisovalo a ukládalo do formátu CSV. Aby nedošlo k přehlcení operační paměti nebo zablokování procesu ze strany serveru knihovny (Rate Limiting),

implementoval jsem mechanismus kontroly velikosti fronty a řízené časové prodlevy mezi jednotlivými požadavky.



Obrázek 2 vývojový diagram vytěžování, vylepšený

2.3 Kontext pro chatbota

Autor minulé práce, Adam Kalina, píše: (1)

- File Search probíhá v rámci backendu a využívá binární vyhledávání či jiné optimalizované metody, aby se asistent dostal k relevantnímu úryvku textu.
- V praxi tak asistent při složitějším dotazu (např. „Jak nastavím maximální denní rozpočet pro marketingovou kampaň?“) vyhledá ve svých souborech odpovídající dokumentaci a lépe dokáže poradit.

I v tomto případě bylo nutné vyřešit zajištění kontextu pro chatbota. Řešení pana Kaliny, autora předchozí práce, však pro tento případ použití není vhodné z následujících důvodů:

- Bylo potřeba hledat v datasetu knihy na základě pole description, což by bylo velice neefektivní při použití jednoduchých algoritmů (binární vyhledávání, porovnávání řetězců)

- U stránky se počítalo s dynamickým měněním, takže by statické soubory nefungovaly. (toto řešení je dále rozvedeno v kapitole 2.6.5)

2.3.1 Kontextové porozumění textu a embeddingy (12) (13) (14)

Pro účely sémantického vyhledávání bylo nezbytné převádět textové informace do numerické reprezentace umožňující porovnávání významu. Tento proces je realizován pomocí tzv. embedding modelů, které mapují text do vysoko dimenzionálního vektorového prostoru.

Na rozdíl od tradičních metod (např. binární vyhledávání nebo porovnávání řetězců) umožňuje embedding zachytit významovou podobnost mezi texty, nikoli pouze jejich doslovnou shodu.

Zjednodušeně lze text reprezentovat jako vektor:

```
Pes => (1, 0, 0.2, 0)
Kočka (1, 1, 0.2, 0)
Lednice => (0, 0, 0.8, 1)
```

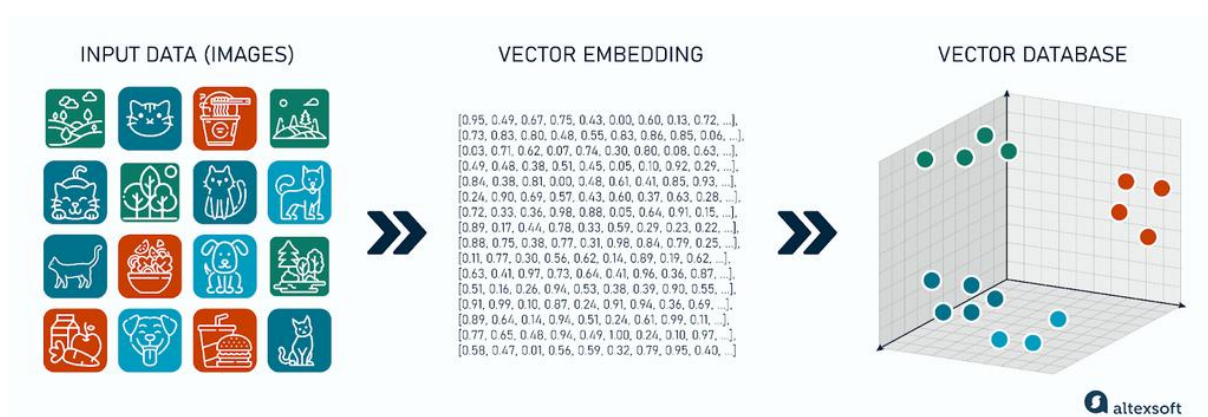
Reálné modely pracují s vektory o stovkách až tisících dimenzí (v této práci 1536). Podobnost mezi dvěma texty je následně určována pomocí kosinové similarity:

$$\text{similarity} = \cos(\theta) = \frac{A \cdot B}{|A||B|}$$

Tento princip umožňuje chatbotovi vyhledávat knihy podle obsahové podobnosti, nikoli pouze podle přesné textové shody.

2.3.2 Vektorová databáze (7)

Vektorová Databáze je relativně nový způsob ukládání dat k účelu získávání podobností. Funguje tak, že z dat, uděláme Embedding, což je vektor, který následně uložíme do databáze. Query provedeme tak, že z hledaného textu uděláme také Embedding. Poté výsledný vektor porovnáme pomocí kosinové similarity (viz vzorec výše) a nejbližších n vektorů vrátíme. Fungování je graficky znázorněno na obrázku níže (obrázek 3).



Obrázek 3 diagram znázorňující fungování vektorových databází (15)

2.3.3 Normalizace CSV v jazyce Python

Analýza získaných dat ukázala výraznou nekonzistenci metadat. Zatímco základní identifikační údaje byly vyplněny téměř ve 100 % případech, pole „Description“ obsahovalo hodnotu pouze u 18,73 % záznamů.

Vzhledem k tomu, že právě popis knihy představuje nejdůležitější zdroj sémantické informace, bylo nutné rozhodnout, které záznamy budou zahrnuty do Embedding procesu. Níže si můžete prohlédnout kompletní dataset:

=== Non-null counts per column ===

Identifier: 937364 (100.00%)

Title: 937364 (100.00%)

Author: 614557 (65.56%)

Contributors: 336574 (35.91%)

Publisher: 833994 (88.97%)

PublicationYear: 830356 (88.58%)

ISBN: 335594 (35.80%)

ISSN: 91152 (9.72%)

Subjects: 554790 (59.19%)

Description: 175555 (18.73%)

Language: 901168 (96.14%)

PhysicalDescription: 885261 (94.44%)

Series: 256258 (27.34%)

Notes: 626856 (66.87%)

RecordType: 937333 (100.00%)

ContentType: 183918 (19.62%)

MediaType: 176836 (18.87%)

CarrierType: 183920 (19.62%)

Následující varianty, které byly testovány:

1. Omezený dataset (~12 000 knih) s kompletním popisem
2. Kombinace Title + (Author nebo Contributors) + Description + Subjects? (~170 000 knih)

Experimentální testování ukázalo, že zahrnutí záznamů bez popisu vede k nárůstu šumu při obecných dotazech, protože algoritmus zvýhodňuje shodu v názvu oproti obsahové podobnosti.

Finální řešení proto zahrnuje pouze záznamy obsahující alespoň *Description*, čímž došlo ke snížení rozsahu databáze, avšak k výraznému zvýšení relevance výsledků.

2.4 Function Calling

*„umělá inteligence si sama řekne o doplňující data“
- Adam Kalina, Marketingový AI Asistent, 2025*

Dovolím si opět citovat fungování mechanismu function calling, jak bylo popsáno v předchozí práci. Princip zůstává totožný i zde – jazykový model je schopen rozpoznat, že k odpovědi potřebuje externí data z backendu, a explicitně si o ně požádá.

(...) vyžadující data z backendu (např. detail o kampani, stav kreditu e-shopu, výpočet statistik atd.), mohl:

- Vyvolat požadavek na volání funkce – tj. vrátit speciální odpověď s informací, že je potřeba zavolat funkci X s parametry Y.
- Vrátit status „RequiresAction“ – aplikace v tento okamžik ví, že má na backendu danou funkci spustit.
- Předat modelu výsledek – asistent pak dostane potřebná data (např. „aktuálně máš kredit 5000 Kč“) a může v konverzaci pokračovat s plným kontextem.

Oproti marketingovému asistentovi, kde bylo implementováno větší množství funkcí, byly v této práci definovány pouze čtyři funkce. Jejich počet je nižší, avšak některé

z nich jsou logicky komplexnější. Architektonicky zde nebyl zaveden nový princip – šlo o adaptaci již ověřeného řešení na jiné doménové prostředí.

Implementované funkce:

- *recommendBooks*
- *findBookByPlot*
- *searchCatalog*
- *searchWebsite*

Funkce *searchCatalog* zajišťuje deterministické vyhledávání podle strukturovaných metadat (např. název nebo autor) a je využívána v situacích, kdy uživatel zná konkrétní titul. Naproti tomu *searchWebsite* poskytuje informace z webového prostředí knihovny (např. otevírací dobu nebo registrační podmínky) a není napojena na knihovní fond.

Z architektonického hlediska je klíčové rozlišení mezi funkcemi *recommendBooks* a *findBookByPlot*. Obě využívají sémantické vyhledávání nad embeddingy, avšak liší se cílem:

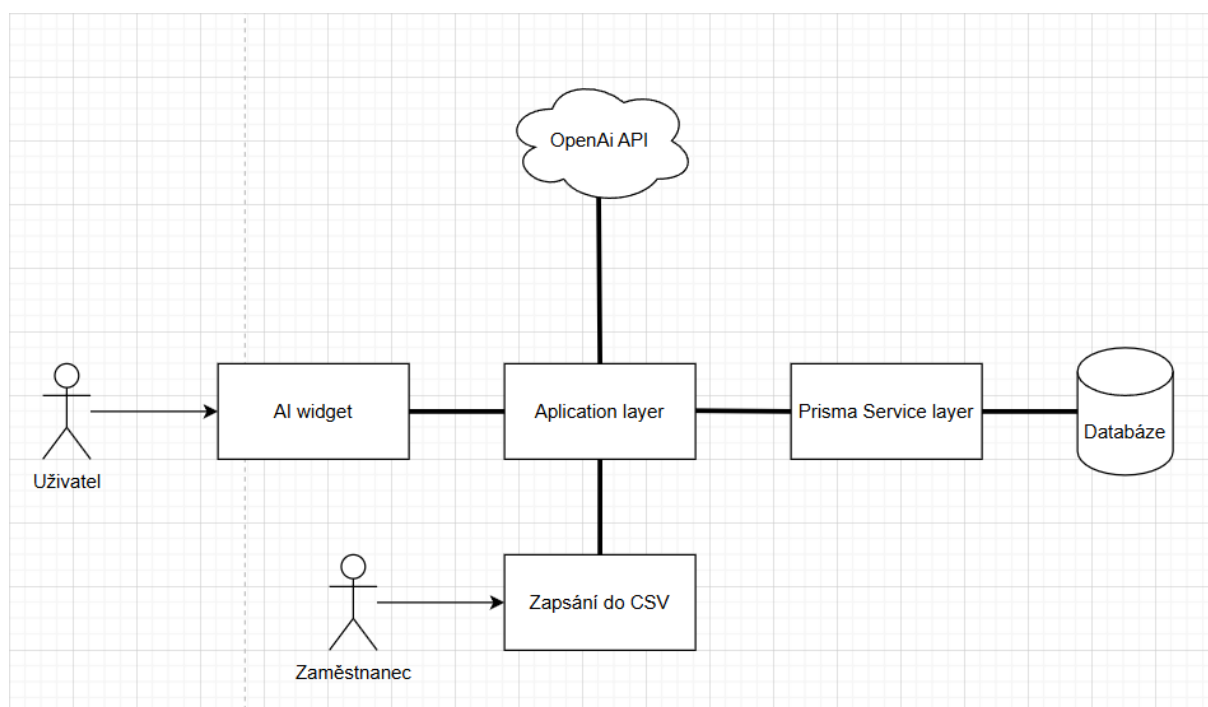
- *recommendBooks* vrací množinu tematicky relevantních titulů,
- *findBookByPlot* usiluje o identifikaci jednoho nejpravděpodobnějšího výsledku na základě popisu děje.

Toto oddělení umožňuje přesnější rozhodování modelu při volání funkcí a jemnější řízení chování asistenta v závislosti na typu uživatelského dotazu.

2.5 Next.js aplikace

Pro realizaci frontendové i backendové části aplikace bylo nutné zvolit vhodný framework. Jako výsledné řešení byl vybrán framework Next.js, který umožňuje vývoj klientské i serverové logiky v rámci jednoho projektu. Toto rozhodnutí přispělo k technologické jednotnosti aplikace a zjednodušilo správu zdrojového kódu. Přestože takový přístup může zvyšovat architektonickou komplexitu, výhodou je centralizace logiky, jednotné prostředí a snazší nasazení.

Současně bylo nutné zvolit další podpůrné knihovny a nástroje, zejména ORM nástroj, databázový systém, UI knihovnu a technologii pro tvorbu API vrstvy. Výsledná architektura aplikace je znázorněna v diagramu (Obrázek 4).



Obrázek 4 diagram architektury aplikace

2.5.1 Frontend (16)

Frontend aplikace je rozdělen na dvě části:

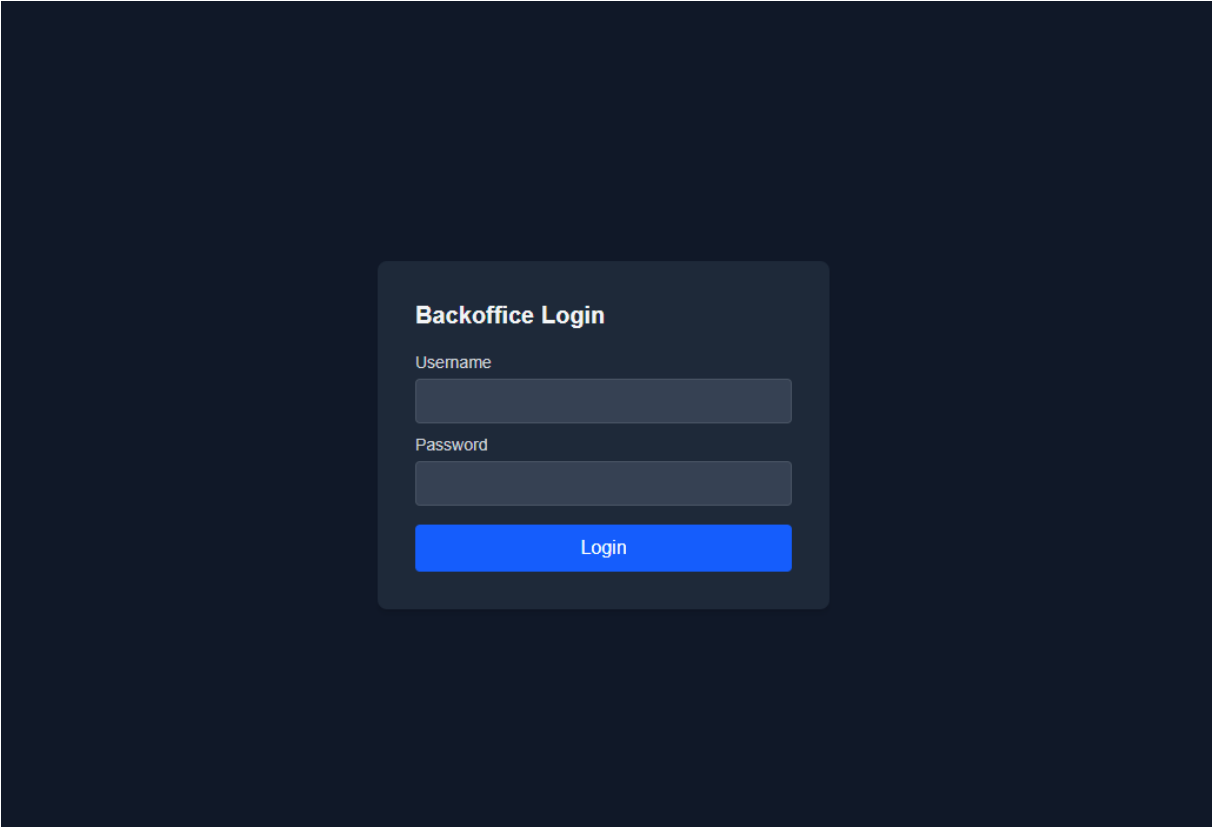
- uživatelský chatovací widget,
- administrátorské rozhraní (backoffice).

Chatovací widget je navržen jako vložitelná komponenta určená pro integraci do webového prostředí knihovny. Klíčovým požadavkem byla izolovanost komponenty, minimální zásah do existujícího DOM prostředí a zachování vizuální konzistence s mateřským webem.

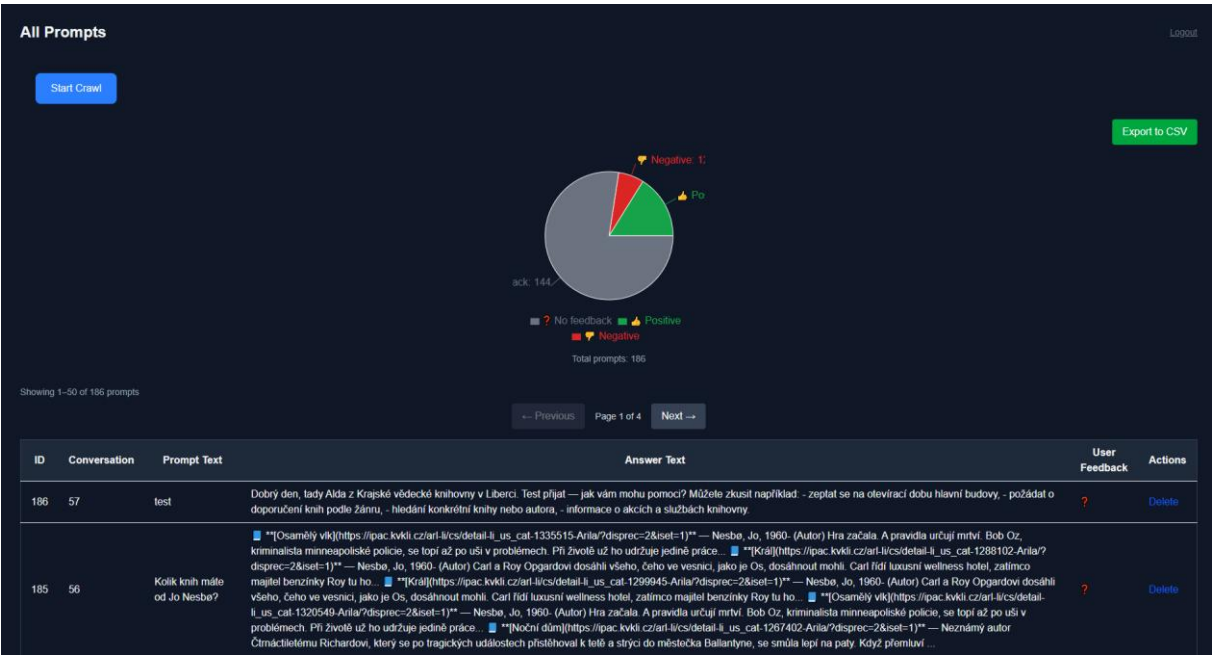
Pro stylování byl zvolen framework Tailwind CSS, který umožňuje deklarativní definici stylů přímo v komponentách a přispívá k rychlejší iteraci vývoje.

V průběhu testování bylo identifikováno chybějící řešení pro stav překročení denního limitu dotazů. Tento stav byl následně doplněn o samostatnou UI komponentu informující uživatele o omezení služby.

Administrátorské rozhraní slouží ke správě promptů, monitorování interakcí a základní kontrole generovaných odpovědí. Prioritou byla funkčnost a rychlá orientace uživatele, nikoli komplexní UI návrh. Na dalších stránkách si můžete prohlédnout design komponenty a backoffice.



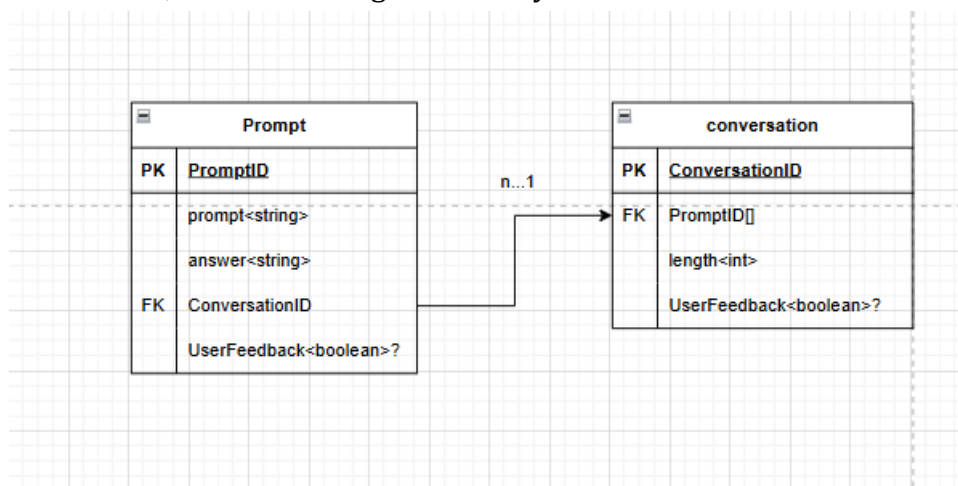
Obrázek 7 design přihlašovací stránky backoffice



Obrázek 8 design backoffice dashboard

2.6 Aplikační architektura

Backendová vrstva zajišťuje perzistenci dat, aplikační logiku a orchestraci komunikace s externí AI službou. Architektura je navržena jako vícevrstvá s oddělením databázové, servisní a integrační vrstvy.



Obrázek 9 třídový diagram databáze

2.6.1 Datová vrstva (Prisma + PostgreSQL) (3)

Pro práci s databází byl zvolen nástroj Prisma ORM, který poskytuje moderní objektově-relační mapování pro prostředí Node.js a TypeScript. Prisma umožňuje definovat datový model prostřednictvím přehledného schématu a následně generovat typově bezpečný klient pro komunikaci s databází.

Jako databázový systém byl využit PostgreSQL, přičemž připojení je realizováno pomocí konfigurační proměnné prostředí. Generátor *prisma-client-js* je nastaven s podporou více binárních cílových platforem, což umožňuje bezproblémové nasazení aplikace v různých provozních prostředích.

Databázové schéma obsahuje dva hlavní modely: *Conversation* a *Prompt*.

Model *Conversation* reprezentuje jednotlivé konverzace a obsahuje primární klíč *conversationId*, číslo určující délku konverzace a volitelné pole pro uživatelskou zpětnou vazbu včetně textové zprávy.

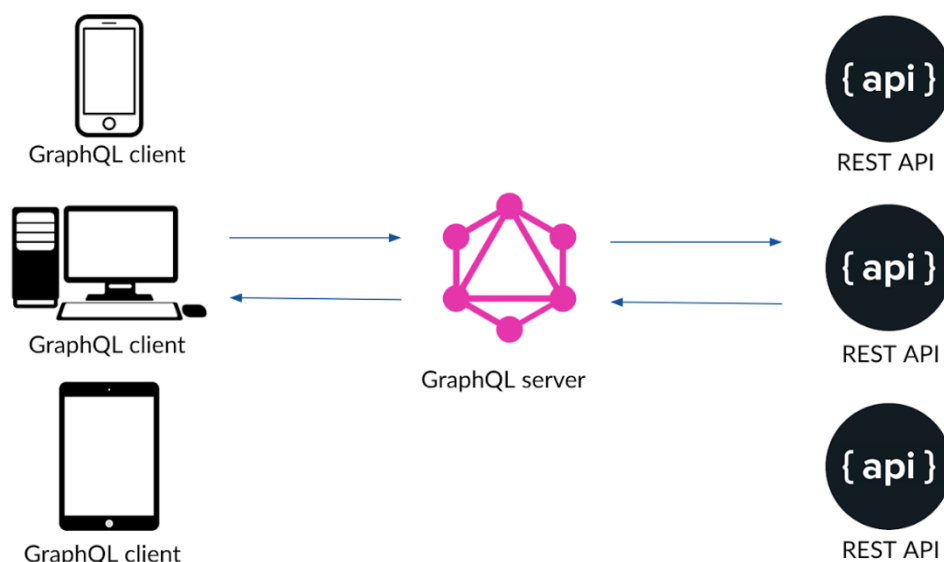
Model *Prompt* reprezentuje jednotlivé dotazy a odpovědi v rámci konverzace. Obsahuje primární klíč *promptId*, text otázky a odpovědi (datový typ řetězec) a cizí klíč *conversationId*, který vytváří relační vazbu na model *Conversation*. Mezi entitami je definován vztah typu jedna-ku-mnohým – jedna konverzace může obsahovat více záznamů typu *Prompt*.

Toto schéma zajišťuje strukturované a konzistentní ukládání dat při zachování typové bezpečnosti na aplikační úrovni.

2.6.2 API vrstva (GraphQL + Apollo) (4)

Pro návrh aplikačního rozhraní byla zvolena technologie GraphQL společně s klientskou knihovnou Apollo Client.

GraphQL představuje alternativu ke klasickému REST přístupu a umožňuje klientovi specifikovat přesnou strukturu požadovaných dat. Tento princip vede k efektivnějšímu přenosu informací a omezuje nadbytečné datové toky.



GraphQL for BE devs | @engfragui

Obrázek 10 jednoduchý diagram graphql technologie (17)

Volba GraphQL však přinesla několik výhod zejména z hlediska typové bezpečnosti, práce s komplexnějšími datovými strukturami a budoucí rozšiřitelnosti systému. Díky silné vazbě mezi GraphQL schématem a TypeScript typy dochází k lepší kontrole konzistence dat mezi klientem a serverem.

V kontextu této práce tak GraphQL nepředstavuje nutnost z hlediska objemu dat či optimalizace přenosu, ale spíše architektonické rozhodnutí směřující k čistší definici aplikačního kontraktu mezi frontendem a backendem.

Knihovna Apollo byla využita pro integraci GraphQL do frontendové části aplikace a poskytuje nástroje pro správu dotazů (Query), mutací (Mutation), cachování a optimalizaci práce s daty.

2.6.3 Aplikační logika (Resolver – Service Architektura)

S rostoucí komplexitou backendu bylo nezbytné oddělit jednotlivé odpovědnosti a zavést přehlednější strukturu kódu. Z tohoto důvodu byla implementována architektura typu resolver–service, která odděluje vrstvu GraphQL resolverů od aplikační logiky.

Vrstva resolverů slouží jako vstupní bod pro GraphQL operace (Query a Mutation). Přijímá požadavky klienta, předává je servisní vrstvě a vrací výsledek zpět do GraphQL schématu. Resolver neobsahuje vlastní aplikační logiku, ale pouze deleguje zpracování na příslušnou službu.

Servisní vrstva (soubory. service.ts) obsahuje implementaci aplikační logiky, práci s databází prostřednictvím Prisma ORM a případnou komunikaci s externí AI službou. Toto oddělení přispívá k lepší čitelnosti, testovatelnosti a rozšiřitelnosti aplikace, protože každá vrstva má jasně definovanou odpovědnost.

```
1 import { promptService } from "../services/prompt.service";
2 import { prismaService } from "../services/prisma.service";
3 import { AddPromptArgs, AddPromptFeedbackArgs } from "../types";
4
5 export const promptResolvers = {
6   Query: {
7     Mutation: {
8       addPrompt: async (_, unknown, args: AddPromptArgs) => {
9         return promptService.addPrompt(args);
10      },
11    },
12  };
13
```

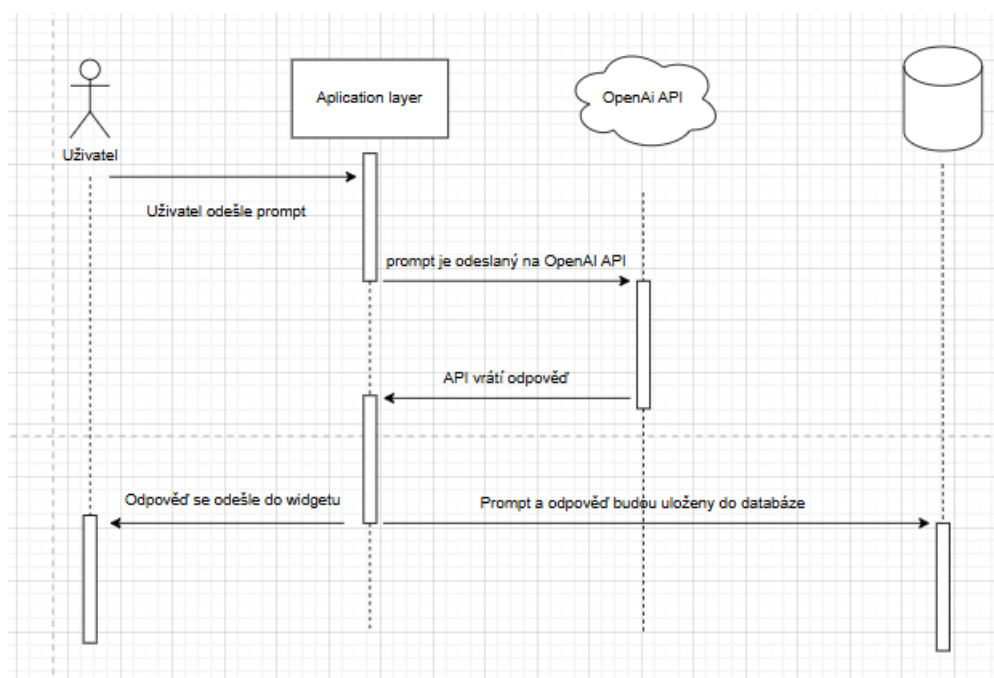
Obrázek 11 obrázek zachycující příkladový model resolveru (codesnap)

2.6.4 Integrace AI

Služba *promptService* zajišťuje aplikační logiku spojenou s ukládáním uživatelských dotazů, generovaných odpovědí a následné zpětné vazby. Pro komunikaci s databází je využíváno Prisma ORM, zatímco generování odpovědi je delegováno na samostatnou AI službu (*aiService*).

Metoda *addPrompt* slouží k vytvoření nového záznamu dotazu. Pokud není předána identifikace existující konverzace, je vytvořen nový záznam entity *Conversation*. Následně je prostřednictvím AI služby vygenerována odpověď na základě vstupního textu a společně s dotazem je uložena do databáze jako nový záznam modelu Prompt. Tím je zajištěna konzistence vztahu mezi konverzací a jednotlivými zprávami.

Metoda *addPromptFeedback* umožňuje ukládání uživatelské zpětné vazby ke konkrétnímu dotazu. Na základě identifikace záznamu je aktualizováno pole *userFeedback*. Proces je doplněn o logování informačních a chybových stavů, což přispívá ke zvýšení robustnosti a dohledatelnosti chování systému.



Obrázek 12 sekvenční diagram požadavku

Služba *aiService* zajišťuje komunikaci s jazykovým modelem společnosti OpenAI a slouží ke generování odpovědí na uživatelské dotazy. Metoda *generateAnswer* nejprve vyhledá relevantní obsah z webu knihovny pomocí funkce *searchSimilarContent*. Získaný kontext je následně vložen do systémové zprávy modelu, čímž je implementován princip RAG, tedy obohacení generované odpovědi o externí znalostní zdroje.

Součástí komunikace s modelem je také definice funkcí *recommendBooks* a *findBookByPlot*. Pokud model vrátí odpověď typu *function_call*, dojde k jejich vyvolání a k následnému sémantickému vyhledání knih prostřednictvím *vectorService*. Výsledky jsou následně transformovány do strukturované textové odpovědi. V případě standardní odpovědi bez volání funkcí je vrácen přímý výstup modelu, případně doplněný o odkazy na zdrojový obsah. Celý proces je monitorován prostřednictvím logovací služby, která zaznamenává průběh operací i případné chybové stavy.

2.6.5 Zpracovávání externích dat (18)

Služba *crawlService* zajišťuje automatizované procházení webových stránek knihovny a extrakci jejich strukturovaného obsahu pro další zpracování. Hlavní metoda *crawlSite* postupně navštěvuje jednotlivé stránky od zadané výchozí adresy, ukládá jejich obsah a identifikuje další relevantní odkazy v rámci stejné domény. Pro zvýšení efektivity je implementováno paralelní zpracování více stránek současně a řízené časové zpoždění mezi dávkami požadavků.

Při zpracování konkrétní stránky je nejprve ověřeno, zda se jedná o relevantní HTML obsah (vyloučeny jsou binární soubory, administrátorské sekce, RSS kanály apod.). Následně je s využitím knihovny ***node-html-parser*** extrahována hlavní obsahová část

stránky (preferenčně element <main> nebo <article>) a text je strukturován do hierarchických sekcí podle nadpisů úrovně h1–h6. Tím vzniká přehledná reprezentace obsahu vhodná pro další zpracování, například generování embeddingů.

Implementace dále obsahuje mechanismus sledování průběhu procesu (stav, počet navštívených stránek, aktuální URL, čas zahájení a ukončení) a možnost jeho předčasného ukončení. Výsledná data jsou ukládána ve formátu JSON do výstupního souboru. Celý proces je doplněn o logování událostí a chybových stavů, což přispívá k jeho robustnosti a kontrolovatelnosti.

Z hlediska efektivity však toto řešení představuje určitou režii zpracování (resource overhead) a potenciální riziko zahrnutí nerelevantního či redundantního obsahu, což může negativně ovlivnit kvalitu generovaných odpovědí.

Alternativní přístup spočívá ve vytvoření specializovaného modelu pro klasifikaci obsahu webu na stěžejní a méně relevantní informace. Takto filtrovaný obsah by následně sloužil jako kvalitnější a přesněji vymezený kontext pro chatbot, namísto plošného procházení celé domény.

Tento přístup by mohl vést ke snížení výpočetní náročnosti i ke zvýšení přesnosti odpovědí, avšak za cenu vyšší implementační complexity a potřeby trénování či ladění specializovaného klasifikačního modelu.

3 Nasazení

Cílem této kapitoly je popsat způsob, jakým byla aplikace nasazena do produkčního prostředí tak, aby byla dlouhodobě dostupná, stabilní a zároveň snadno aktualizovatelná. Nasazení muselo řešit nejen samotnou aplikaci vytvořenou pomocí frameworku Next.js, ale také databázovou vrstvu, vektorovou databázi a reverzní proxy server.

3.1 VPS

V průběhu vývoje bylo nutné vytvořit prostředí, ve kterém bylo možné průběžně testovat nové verze aplikace. Zároveň toto prostředí umožňovalo jednodušší získávání zpětné vazby od uživatelů. Namísto osobních konzultací bylo možné zpřístupnit běžící instanci aplikace prostřednictvím IP adresy serveru, což umožnilo testerům samostatně procházet jednotlivé funkce aplikace na vlastním zařízení.

Původně bylo zvažováno nasazení pomocí **managed hostingových služeb** (např. Vercel¹) nebo interního školního serveru pluto.pslib.cz. Tato varianta však neumožňovala provoz vlastních databázových instancí ani vektorové databáze v požadované konfiguraci. Z tohoto důvodu bylo zvoleno řešení formou pronájmu virtuálního privátního serveru (VPS), které poskytuje plnou kontrolu nad operačním systémem a sítíovou konfigurací.

3.1.1 Dockerfile a Docker Compose

Pro zajištění konzistentního běhu aplikace napříč vývojovým a produkčním prostředím byla celá infrastruktura postavena na technologii Docker. Kontejnerizace umožnila oddělit jednotlivé části aplikace, zjednodušit nasazení a minimalizovat problémy způsobené rozdíly v konfiguraci prostředí.

3.1.1.1 Dockerfile

Pro optimalizaci výsledné velikosti obrazu byl použit multi-stage build². První fáze slouží k instalaci závislostí, druhá k sestavení aplikace a třetí představuje minimalizované produkční prostředí.

3.1.1.2 Docker Compose (produkční konfigurace)

Pro orchestraci jednotlivých služeb byl použit nástroj Docker Compose. Ten umožňuje spustit více kontejnerů současně a definovat mezi nimi vazby.

¹ cloudová platforma pro nasazení a provoz webových aplikací

² technika sestavení kontejneru využívající více fází pro minimalizaci výsledného obrazu

Celá aplikace se skládá z následujících služeb:

- app – hlavní Next.js aplikace, která komunikuje s databází PostgreSQL a vektorovou databází ChromaDB
- db – PostgreSQL databáze sloužící pro ukládání aplikačních dat
- chromadb – vektorová databáze pro ukládání embeddingů knih
- nginx – reverzní proxy server, který zajišťuje přístup k aplikaci ve veřejné síti

Služba app je závislá na dostupnosti databáze a ChromaDB, což je řešeno pomocí healthchecků. Tím je zajištěno, že aplikace se spustí až ve chvíli, kdy jsou všechny závislé služby připravené.

PostgreSQL i ChromaDB využívají persistentní volumes, aby nedocházelo ke ztrátě dat při restartu kontejnerů. Nginx je využit jako vstupní bod do aplikace a přesměrovává HTTP provoz na Next.js server běžící uvnitř Docker sítě.

Architektura je navržena jako izolovaná kontejnerová síť, ve které jsou jednotlivé služby dostupné pouze v rámci interní Docker sítě, zatímco veřejný přístup je směřován výhradně přes reverzní proxy server Nginx.

Použití Docker Compose výrazně zjednodušilo správu celého nasazení, umožnilo rychlé restartování služeb a zajistilo opakovatelnost prostředí.

3.1.2 CI/CD pipelines

Pro zjednodušení a automatizaci nasazování byly vytvořeny CI/CD pipelines pomocí služby GitHub Actions. Tyto pipelines se spouští automaticky při každém git push do hlavní větve repozitáře a zajišťují kompletní proces od ověření funkčnosti kódu až po jeho nasazení na produkční server. Díky tomu je minimalizováno riziko chyb způsobených manuálním nasazováním a zároveň je výrazně zrychlen celý vývojový cyklus.

Pipeline je rozdělena do dvou hlavních částí – testovací a nasazovací.

3.1.2.1 Testovací fáze

První fáze pipeline slouží k ověření, zda aplikace je ve funkčním stavu a nově přidané změny nerozbíjejí existující funkcionality. Tato část běží na izolovaném prostředí typu ubuntu-latest.

V rámci této fáze dojde k:

- stažení zdrojového kódu z repozitáře,
- nastavení správné verze Node.js,
- instalaci závislostí pomocí npm ci,
- spuštění unit testů.

Pokud kterýkoli krok v této fázi selže, pipeline se ukončí a k nasazení nedojde. Tím je zajištěno, že na produkční server se dostane pouze otestovaný kód.

3.1.2.2 Nasazovací fáze

Nasazovací fáze se spustí pouze v případě, že testovací fáze proběhne úspěšně. Tato část pipeline zajišťuje samotné nasazení aplikace na VPS server.

Připojení k serveru probíhá pomocí SSH klíče, který je bezpečně uložen v GitHub Secrets. Server je nejprve přidán do seznamu známých hostitelů, aby bylo možné se k němu připojit bez manuálního potvrzení.

Následně pipeline:

- ověří existenci cílového adresáře na serveru,
- synchronizuje aplikační soubory pomocí nástroje rsync,
- vytvoří konfigurační soubor **.env** přímo na serveru ze zabezpečených proměnných,
- zastaví a odstraní běžící kontejnery včetně tzv. orphan kontejnerů,
- znovu sestaví Docker obrazy a spustí všechny služby pomocí Docker Compose.
- Provede kontrolu stavu služeb a migrace databáze

Po spuštění kontejnerů pipeline aktivně kontroluje zdravotní stav klíčových služeb, zejména databáze PostgreSQL a vektorové databáze ChromaDB. Tento krok je důležitý, protože aplikace je na těchto službách přímo závislá a jejich nedostupnost by vedla k nefunkčnímu nasazení.

V případě problémů pipeline vypisuje detailní logy, které umožňují rychlou diagnostiku chyby. Pokud se služby do určitého časového limitu nespustí ve zdravém stavu, nasazení je považováno za neúspěšné.

Po úspěšném startu služeb jsou spuštěny databázové migrace pomocí nástroje Prisma, čímž je zajištěno, že struktura databáze odpovídá aktuální verzi aplikace. Nakonec jsou spuštěny integrační testy, které ověřují správnou komunikaci aplikace s vektorovou databází.

3.1.2.3 Přínos CI/CD řešení

Zavedení CI/CD pipeline výrazně zjednodušilo celý proces nasazování aplikace. Zavedení CI/CD pipeline minimalizuje riziko manuálních chyb a zajišťuje reprodukovatelnost nasazovacího procesu, má okamžitou zpětnou vazbu o stavu aplikace a může nasazovat nové verze prakticky jedním commitem. Pipeline zároveň slouží jako forma dokumentace celého nasazovacího procesu.

3.2 Produkce

Produkční server knihovny běží na operačním systému Ubuntu 20.04.1 LTS. Na serveru bylo nutné upravit verzi Docker Engine z důvodu kompatibility s verzí jádra operačního systému.

Samotná aplikace je na serveru sestavena pomocí Dockeru a následně spuštěna jako kontejnerová služba.

3.2.1 První testovací kolo

První testovací kolo proběhlo přibližně měsíc před odevzdáním práce. Aplikace nasazená na VPS byla zpřístupněna omezené skupině testerů tvořené spolužáky, známými a zaměstnanci knihovny. Cílem bylo získat první reálnou zpětnou vazbu od potenciálních uživatelů.

Shromážděné dotazy byly následně rozděleny do tří kategorií: úspěšné odpovědi, neúspěšné odpovědi a dotazy, které nebylo možné vyhodnotit. Tyto nevyhodnotitelné dotazy byly z další analýzy vyřazeny.

Celkem bylo v tomto kole shromážděno 130 dotazů. Výsledná míra úspěšnosti odpovědí činila přibližně 65 %. Přestože se tento výsledek může zdát relativně nízký, testování poskytlo cenné informace o chování systému a odhalilo řadu problémů, které by se v laboratorním prostředí obtížně identifikovaly.

3.2.2 Fungování stránky kvkli.cz (19) (20) (21) (22)

Webové stránky knihovny kvkli.cz jsou provozovány na vlastním redakčním systému napsaném v jazyce **PHP**. Tento systém využívá šablonovací nástroj **Smarty** pro generování HTML výstupu a ukládá data v databázi MySQL. Frontendová část využívá knihovnu **jQuery** a editor **CKEditor** pro správu obsahu.

Z pohledu této práce je důležité především to, že systém umožňuje vkládání vlastních JavaScriptových skriptů do šablon stránky. Díky tomu bylo možné do existujícího webu integrovat chatbot formou externího widgetu bez nutnosti zásahů do jádra aplikace.

3.2.3 Widget

Integrace chatbotu do webových stránek byla realizována formou samostatného JavaScriptového widgetu. Tento widget obsahuje kompletní klientskou logiku chatbotu a je distribuován jako jeden kompilovaný soubor widget.js.

Soubor byl následně umístěn do adresáře

`kvkli/sites/kvkli/js`

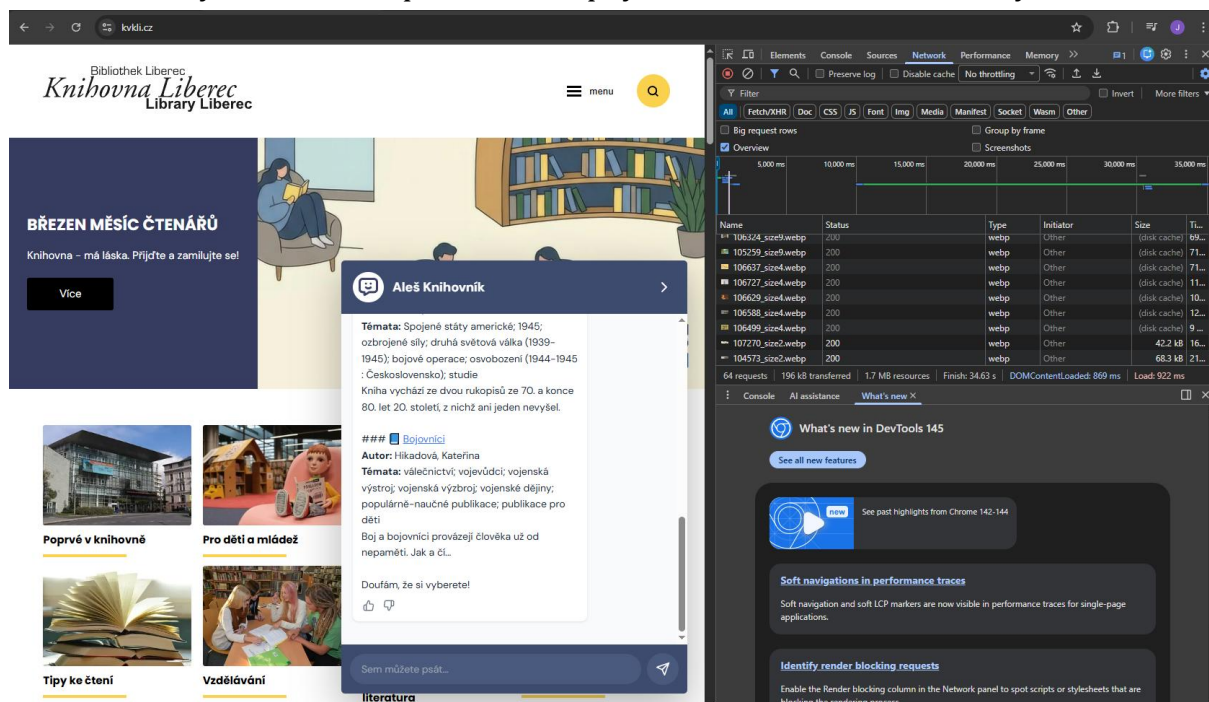
a načten v patičce hlavní stránky pomocí následujícího skriptu:

```
<script type="text/javascript" src="{$_SITE/js/widget.js|fileversion}"></script>
```

Obrázek 13 script pro nasazení widget (ChatGPT)

3.2.4 Následné nasazení

První nasazení chatbotu na produkční web knihovny proběhlo dne 6. 3. 2026. Widget byl na stránku přidán v přibližně 2:30 ráno, aby bylo minimalizováno případné ovlivnění běžného provozu webu. Nasazení proběhlo bez technických komplikací a chatbot byl krátce poté dostupný návštěvníkům webových stránek.



Obrázek 14 obrázek ukazující nasazení AI widget na produkční web

3.2.5 Budoucnost nasazování

Do budoucna je plánováno vytvoření samostatné CI/CD pipeline pro automatické nasazování produkční verze chatbotu. Součástí tohoto řešení bude také vytvoření samostatné subdomény chatbot.kvkli.cz, na které bude hostována backendová část aplikace. Implementace tohoto řešení však již přesahuje rámec této práce.

4 Bezpečnost

Tato kapitola popisuje bezpečnostní mechanismy implementované v aplikaci. Jejich cílem je omezit možnosti zneužití systému, zejména neoprávněné využívání API, nadměrné zatěžování služby nebo pokusy o manipulaci s aplikační logikou.

Přestože aplikace nepracuje s citlivými osobními údaji, bylo nutné implementovat základní bezpečnostní opatření odpovídající běžným standardům webových aplikací.

4.1 Bezpečnostní řešerše

V první fázi návrhu bylo nutné identifikovat potenciální bezpečnostní rizika a určit, které části systému vyžadují ochranu. Analýza se zaměřila zejména na možné zneužití API rozhraní, nadměrné zatěžování služby automatizovanými dotazy a neoprávněný přístup do administračního rozhraní.

4.1.1 Middleware + CORS (23)

První implementovaný bezpečnostní mechanismus představuje **middleware** komponenta umístěná v souboru:

```
/app/graphql/middleware/originGuard.ts
```

Jejím účelem je omezení přístupu k API prostřednictvím mechanismu **CORS** (Cross-Origin Resource Sharing). Middleware kontroluje hlavičku Origin příchozích požadavků a povoluje komunikaci pouze z předem definovaného seznamu důvěryhodných domén.

Tento přístup využívá princip tzv. bílé listiny (whitelist), ve které jsou uvedeny webové stránky oprávněné komunikovat s API. Požadavky přicházející z jiných domén jsou odmítnuty ještě před zpracováním aplikační logiky.

Cílem tohoto opatření je zamezit neoprávněnému využívání API třetími stranami.

4.1.2 Prompt limit cookies (24)

Dalším implementovaným mechanismem je omezení maximálního počtu dotazů v rámci jedné konverzace. Cílem tohoto opatření je zabránit nadměrnému využívání jazykového modelu a zbytečnému spotřebovávání tokenů.

Kontrola limitu je realizována na serverové straně. Při každém novém dotazu je aktuální počet promptů porovnán s konstantou:

```
MAX_PROMPTS_PER_CONVERSATION
```

Po překročení tohoto limitu je do prohlížeče uživatele uložena **cookie**, která informuje klientskou aplikaci o dosažení limitu a zajišťuje perzistenci tohoto stavu i po opětovném načtení stránky.

Řešení poskytuje základní úroveň ochrany proti běžnému zneužití. Proti cílenému obcházení však neposkytuje plnou ochranu, a proto je do budoucna plánována robustnější implementace.

4.1.3 Zabezpečení administrativního rozhraní

Přestože aplikace neukládá citlivá data, je standardní praxí chránit administrativní rozhraní přístupovou autentizací. Backoffice slouží k monitorování konverzací, správě promptů a analýze uživatelské zpětné vazby, a proto by neměl být veřejně dostupný.

Přístup do administrační části je proto chráněn jednoduchým přihlašovacím mechanismem založeným na statických přihlašovacích údajích. Tyto údaje jsou načítány z konfiguračního souboru .env a nejsou součástí zdrojového kódu aplikace.

4.1.4 Heartbeat endpoint

Dalším implementovaným prvkem je tzv. **heartbeat endpoint** sloužící jako jednoduchý mechanismus kontroly dostupnosti služby. Chatovací widget při inicializaci ověřuje dostupnost backendu pomocí HTTP požadavku na tento endpoint.

Pokud server vrátí stavový kód 200, widget se zobrazí a umožní uživateli zahájit konverzaci. V opačném případě se komponenta nenačte.

Tento mechanismus slouží jako ochrana proti situacím, kdy backend není dostupný. Z pohledu uživatele tak nedochází k zobrazování nefunkčního rozhraní nebo chybových hlášení.

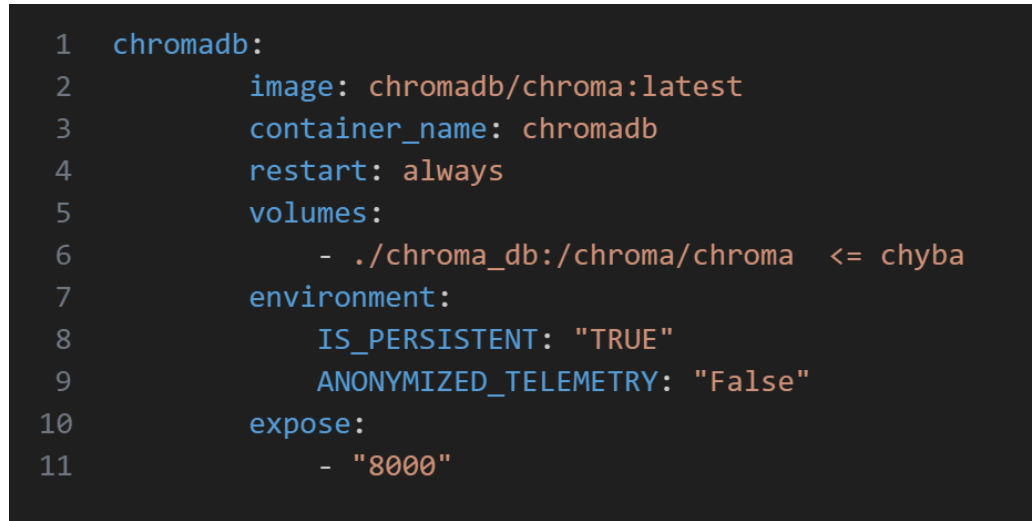
5 Identifikované problémy a jejich řešení

Tato kapitola shrnuje klíčové technické problémy identifikované během návrhu a implementace systému. Jejich analýza přispívá k lepšímu pochopení zvolených architektonických rozhodnutí a může sloužit jako metodická opora pro obdobné projekty.

5.1 Kontejnerizační a konfigurační problémy

V průběhu vývoje docházelo k opakovaným selháním při spouštění služby ChromaDB v rámci konfigurace Docker Compose. Příčinou byla nesprávně definovaná mapovaná cesta pro persistentní úložiště (`./chroma_db:/chroma/chroma`), která neodpovídala interní adresářové struktuře kontejneru.

Důsledkem bylo neukládání embedding dat a ztráta indexu po restartu služby. Řešení spočívalo v úpravě cílové cesty dle oficiální dokumentace a ověření správného mapování prostřednictvím inspekce běžícího kontejneru pomocí nástroje `docker exec`.



```
1 chromadb:
2   image: chromadb/chroma:latest
3   container_name: chromadb
4   restart: always
5   volumes:
6     - ./chroma_db:/chroma/chroma <= chyba
7   environment:
8     IS_PERSISTENT: "TRUE"
9     ANONYMIZED_TELEMETRY: "False"
10  expose:
11    - "8000"
```

Obrázek 15 obrázek zachycující Docker Compose chybu (codesnap)

5.2 Problémy integrace vektorové databáze

V rámci testovací fáze bylo experimentováno s cloudovou variantou databáze ChromaDB. Při integraci došlo k problému nekompatibility rozměrů embedding vektorů (dimension mismatch). Embedding model generoval vektory o odlišné dimenzi, než jaká byla inicializována v databázové kolekci. Tento nesoulad vedl k selhání operací ukládání a vyhledávání.

Řešením bylo sjednocení dimenzionality modelu a databázové kolekce a opětovná inicializace indexu.

5.3 Multijazyčné prostředí projektu

Použití tří odlišných technologických stacků (C# pro scraper, Python pro datovou přípravu a TypeScript pro webovou aplikaci) vedlo ke zvýšené komplexitě správy projektu. Každá část vyžadovala samostatné prostředí, závislosti a konfigurační mechanismy, což komplikovalo návrh CI/CD pipeline.

Přestože tento přístup umožnil využít specializované nástroje v jednotlivých fázích vývoje, z hlediska dlouhodobé udržitelnosti by bylo vhodnější sjednotit aplikační logiku do jednoho ekosystému, například do prostředí TypeScript/Node.js.

5.4 Problematika RAG struktury

Nejvíce koncepčních problémů se objevilo při návrhu a implementaci architektury typu RAG.

5.4.1 RAG struktura v katalogu

Hlavním problémem první verze bylo podcenění předzpracování dat. Duplicitní záznamy, nekonzistentní formátování a chybné kódování znaků v CSV souborech způsobovaly degradaci embedding reprezentací. Model následně generoval méně kvalitní vektorové reprezentace, což se negativně projevovalo při dotazech typu „Doporuč mi knihy o hudbě“.

5.4.2 RAG struktura ve webové stránce

Při procházení webu kvkli.cz docházelo k zahrnování navigačních a provozních prvků (menu, patička, systémové texty) do embedding datasetu. Výsledkem bylo, že asistent vracel odpovědi obsahující nerelevantní textové fragmenty. Problém byl vyřešen selektivním parsováním HTML struktury a explicitním filtrováním nerelevantních elementů.

Další problém se týkal struktury jednotlivých stránek. Některé obsahovaly vysoce podobný textový obsah, což vedlo k sémantické nejednoznačnosti při vyhledávání. Dotaz „Řekni mi něco o výpůjčkách“ byl například přiřazován k nerelevantním stránkám s podobnou terminologií.

Pro zvýšení přesnosti byla implementována jednoduchá forma dotazové expanze (query expansion), kdy byl původní uživatelský dotaz rozšířen o doplňující kontextové instrukce. Tím došlo ke zvýšení sémantické specifity vyhledávání.

Specifický problém představovalo i formátování kontaktů. Kontaktní údaje byly při embedování reprezentovány jako jeden souvislý textový řetězec, což znemožňovalo správné přiřazení osob k rolím a telefonním číslům. Namísto implementace vlastního

klasifikačního modelu bylo zvoleno pragmatické řešení ve formě statického strukturovaného TypeScript objektu obsahujícího kontaktní data.

5.4.3 RefaktORIZACE³ A STABILIZACE SYSTÉMU

V závěrečné fázi projektu byla provedena refaktORIZACE částí aplikace. Cílem bylo oddělení odpovědností mezi vrstvami systému, sjednocení práce s chybovými stavy a centralizace konfiguračních parametrů. Tyto kroky vedly ke zvýšení stability systému a zlepšení jeho dlouhodobé udržitelnosti. Projekt je i po odevzdání aktivně udržován a průběžně refaktorován, aby bylo možné reagovat na nové požadavky, optimalizovat výkon a zajišťovat kvalitní uživatelskou zkušenost.

³ RefaktORIZACÍ se rozumí proces úpravy zdrojového kódu tak, aby byl lépe strukturovaný, čitelnější a udržitelnější, aniž by se změnilo jeho chování.

6 Budoucnost této práce

V této kapitole budou popsány možné další postupy a funkcionality, kterými by se mohla tato práce ubírat.

- Přejít na lokální model

Jednou z perspektivních možností je experimentální nasazení lokálního open-source modelu (např. architektury LLaMA). Takový přístup by umožnil snížit závislost na externím poskytovateli API a potenciálně optimalizovat provozní náklady. Současně by však vyžadoval řešení výpočetních nároků, správy modelu a jeho průběžné aktualizace.

- Hlubší integrace s knihovním katalogem

Významným krokem by byla integrace systému s katalogem ipac.kvkl.cz. Propojení autentizačních mechanismů a uživatelských účtů by umožnilo personalizované odpovědi (např. doporučení na základě historie výpůjček nebo upozornění na dostupnost konkrétních titulů). Tato integrace by však vyžadovala zásah do stávající architektury knihovního systému a důkladné řešení bezpečnostních aspektů.

- Inteligentní klasifikace obsahu webu

Jak již bylo řečeno, dal by se místo algoritmu *crawl service* vytrénovat model na rozhodování důležitosti textů a na postavení struktury dat, které by šly dobře indexovat do vektorové databáze. Tím by se do vektorové databáze nedostaly nedůležité informace nebo by minimálně získaly nižší bodové ohodnocení než jiné, třeba otevírací doba, kontakty atd.

- Rozšíření funkce přístupnosti

Další rozvoj může zahrnovat podporu vícejazyčných mutací, integraci technologie převodu textu na řeč (text-to-speech⁴) nebo obráceně (speech-to-text)⁵ nebo další prvky zvyšující dostupnost služby pro širší skupinu uživatelů.

- Konfigurační stránka v backoffice

Posledním vývojem by mohlo jít o vytvoření konfigurační stránky pro zaměstnance, jednoduché přepínání modelu, maximálnímu množství promptů na konverzace, úplné vypnutí chatbota nebo konfigurace odesílání mailů při zachycení erroru.

⁴ Text to speech – TTS, se rozumí syntéza textu na mluvené slovo.

⁵ Speech to text – STT, se rozumí transkripce mluvené řeči na text.

Závěr

Cílem této práce bylo navržení a implementace AI asistenta pro web kvkli.cz. Řešení vycházelo z práce Marketingový AI asistent a osobních poznatků autora o umělé inteligenci a zpracování přirozeného jazyka. Pro implementaci byly využity moderní frameworky a architektury, včetně GraphQL, Docker Compose, embeddings a metody RAG.

Celkově tato práce přispívá k pochopení možností AI asistentů pro znalostně náročné úkoly a poskytuje výchozí bod pro další rozvoj inteligentních webových služeb na kvkli.cz.

Zhodnocení cílů a autorovy poznámky

Všechny cíle této práce byli splněny až na nasazení na školní server, jako vývojového prostředí, to bylo zdůvodněno v kapitole 3.1.

Na této práci bylo stráveno odhadem okolo 150 h – 170 h. Nejobjemnější částí této práce bylo samotné programování. To trvalo asi 90 hodin. Na práci se serverem, pipelines a nasazováním bylo stráveno do 20 hodin. Na dokumentu byla odvedena práce okolo 30 hodin. Schůzky a konzultace odhaduji na 3 hodiny. Průzkumem technologií bylo sečteno zhruba na 10 hodin. Dále na práci byl strávený čas asi k dalším 10 hodinám na designu, testování a plánování.

Práci se budu věnovat i v budoucnu.

Seznam zkratek a odborných výrazů

CI/CD

Continuous Integration / Continuous Deployment – proces automatizované integrace, testování a nasazování kódu.

CSV

Comma-Separated Values – textový formát pro ukládání tabulkových dat oddělených čárkou.

Docker

Platforma pro kontejnerizaci aplikací umožňující jejich izolovaný a přenositelný běh.

Docker Compose

Nástroj pro orchestraci více Docker kontejnerů pomocí konfiguračního souboru.

Embedding

Numerická reprezentace textu ve formě vektoru používaná pro výpočet sémantické podobnosti.

OAI-PMH – The Open Archives Initiative Protocol for Metadata Harvesting

Protokol pro vytěžování metadat mezi systémy.

RAG – retrieval augmented generation

Metoda kombinující vyhledávání relevantních dat s generováním odpovědi jazykovým modelem.

LLM – Large Language Model

Velký jazykový model trénovaný na rozsáhlých datech pro generování textu a pochopení jazyka.

CORS – Cross-Origin Resource Sharing

Mechanismus webových prohlížečů, který umožňuje kontrolovaný přístup k prostředkům (API) z jiných domén, než je původní server.

Managed hostingová služba

Hosting, kde poskytovatel spravuje infrastrukturu, aktualizace a provoz serveru.

VPS – virtual private server (virtuální privátní server)

Virtualizovaný server poskytující dedikované systémové prostředky v rámci sdílené infrastruktury.

Backend

Část systému zajišťující logiku aplikace, zpracování dat a komunikaci s databází.

Frontend

Část systému sloužící jako komunikace mezi uživatelem a backendem.

Tech stack (technologický stack)

Soubor technologií použitých při vývoji provozu aplikace.

Server-side rendering (SSR / vykreslování na straně serveru)

Generování HTML obsahu stránky na serveru před odesláním klientovi.

API (Application Programming Interface)

Rozhraní umožňující komunikaci a výměnu dat mezi softwarovými systémy.

Unit/Jednotkové testy

Automatizované testování jednotlivých částí kódu.

Smoke test

Základní ověření funkčnosti systému po sestavení.

Reverse proxy (reverzní proxy)

Se rozumí server zprostředkující komunikaci mezi klientem a interními aplikačními servery

LLaMA (Large Language Model Meta AI)

Velký jazykový model vyvinutý společností Meta pro zpracování přirozeného jazyka.

Cookie (HTTP cookie)

Malý datový soubor ukládaný v prohlížeči pro uchování informací o relaci uživatele.

Endpoint

Koncový bod API určený pro přístup ke konkrétní službě nebo datům.

Heartbeat

Periodický signál ověřující dostupnost a funkčnost služby.

.env (environment file)

Soubor obsahující konfigurační proměnné prostředí aplikace.

Middleware

Mezivrstva zpracovávající požadavky mezi klientem a aplikační logikou.

Origin

Zdroj webového požadavku definovaný kombinací protokolu, domény a portu.

Widget

Vložitelná komponenta poskytující specifickou funkčnost v uživatelském rozhraní.

PHP

Skriptovací programovací jazyk určený pro vývoj webových aplikací.

jQuery

JavaScriptová knihovna zjednodušující manipulaci s DOM a práci s událostmi.

Smarty

Šablonovací systém pro PHP oddělující prezentační vrstvu od aplikační logiky.

CKEditor

Webový WYSIWYG editor pro tvorbu a úpravu formátovaného obsahu.

Function calling

Rozumí se tím možnost AI modelu zavolat si funkce doplňující kontext.

XML:MARC

standardizovaný formát pro reprezentaci a výměnu knihovních bibliografických záznamů (typicky MARC 21) v jazyce XML.

Seznam obrázků

Obrázek 1 vývojový diagram vytěžování	7
Obrázek 2 vývojový diagram vytěžování, vylepšený	8
Obrázek 3 diagram znázorňující fungování vektorových databází (15)	10
Obrázek 4 diagram architektury aplikace	13
Obrázek 5 schválený design widgetu	14
Obrázek 6 ukázka widgetu s limitem	14
Obrázek 7 design přihlašovací stránky backoffice	15
Obrázek 8 design backoffice dashboard	15
Obrázek 9 třídový diagram databáze	16
Obrázek 10 jednoduchý diagram GraphQL technologie (17)	17
Obrázek 11 obrázek zachycující příkladový model resolveru (codesnap)	18
Obrázek 12 sekvenční diagram požadavku	19
Obrázek 13 script pro nasazení widget (ChatGPT)	24
Obrázek 14 obrázek ukazující nasazení AI widget na produkční web	25
Obrázek 15 obrázek zachycující Docker Compose chybu (codesnap)	28
Obrázek 16 QR na odkaz na github repozitář	I

Použité zdroje

1. **Kalina, Adam.** *Marketingový ai assistant*. Liberec : Pslib, 2024/2025.
2. **Contributors, Next.js.** Next.js Documentation. *Vercel*. [Online] Vercel. [Citace: 6. 3 2026.] <https://nextjs.org/docs>.
3. **Prisma Data, Inc.** Prisma Doc. *Prisma*. [Online] Prisma Data, Inc. [Citace: 3. 1 2026.] <https://www.prisma.io/docs/>.
4. **Apollo Graph Inc.** Apollo GraphQL Docs. *Apollo*. [Online] Apollo Graph Inc. [Citace: 3. 1 2026.] <https://www.apollographql.com/docs/>.
5. **The GraphQL Foundation.** Learn GraphQL. *GraphQL*. [Online] The GraphQL Foundation. [Citace: 3. 1 2026.] <https://graphql.org/learn/>.
6. **Fowler, Martin.** Unit Test. *martinfowler*. [Online] Martin Fowler. [Citace: 9. 3 2026.] <https://martinfowler.com/bliki/UnitTest.html>.
7. **DOKCER INC.** Docker docs. *Docker*. [Online] DOKCER INC. [Citace: 28. 2 2025.] <https://docs.docker.com/>.
8. **Chroma Core team.** Introduction – Chroma Docs. *Chroma Docs — Documentation for the Chroma vector database*. [Online] Chroma Core. [Citace: 3. 1 2026.] <https://docs.trychroma.com/docs/overview/introduction>.
9. **OpenAI, Inc.** Overview | OpenAI Platform. *OpenAI Platform Documentation*. [Online] OpenAI, Inc. [Citace: 3. 1 2026.] <https://platform.openai.com/docs/overview>.
10. **LAGOZE, Carl, et al.** The Open Archives Initiative Protocol for Metadata Harvesting. <https://www.openarchives.org/>. [Online] openarchives, 6. 3 2002. [Citace: 28. 2 2026.] <https://www.openarchives.org/OAI/openarchivesprotocol.html>.
11. **Cosmotron s.r.o.** *Interní firemní dokument - OAI:Poskytování záznamů vzdáleným harvesterům - OAI*. [pdf] online : Cosmotron s.r.o.
12. **Stehlík, Michal.** Transformery. [PowerPoint presentation] Dostupné na: https://pslib.sharepoint.com/:p:/r/sites/studium/it/uin/_layouts/15/Doc.aspx?sourcedoc=%7B5DAE7F78-1A6F-4C40-931F-69ABB918766C%7D&file=Transformer.pptx&action=edit&mobileredirect=true.
místo neznámé : Střední průmyslová škola .
13. **VASWANI, Ashish.** Attention Is All You Need. *Advances in Neural Information Processing Systems*. [Online] NeurIPS, 2017. [Citace: 3. 5 2026.] https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
14. **LEWIS, Patrick, Ethan PEREZ, Aleksandra PIKTUS, Fabio PETRONI, Vladimir KARPUKHIN, Naman GOYAL, Heinrich KÜTTLER, Mike LEWIS, Wen-tau YIH, Tim ROCKTÄSCHEL, Sebastian RIEDEL a Douwe KIELA.** Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems* 33. [Online] 2020. [Citace: 6. 3 2026.]

-
- https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf.
15. **NGUYEN, Linda.** What Is a Vector Database? An Illustrative Guide. *alexsoft*. [Online] 29. 1 2025. [Citace: 10. 3 2026.] <https://www.altexsoft.com/blog/vector-database/>.
 16. **TAILWIND LIBS.** Tailwind css docs. *tailwind css*. [Online] Tailwind css contributors. [Citace: 9. 3 2026.] <https://tailwindcss.com/docs/installation/using-vite>.
 17. **Advice from a GraphQL Expert.** *Netlify Blog*. [Online] GUI, Enrique, 21. 1 2020. [Citace: 4. 3 2026.] <https://www.netlify.com/blog/2020/01/21/advice-from-a-graphql-expert/>.
 18. **TAOEN.** node-html-parser readme. *npmjs*. [Online] npm, Inc. [Citace: 9. 3 2026.] <https://www.npmjs.com/package/node-html-parser>.
 19. **Smarty documentation.** *Smarty*. [Online] SMARTY contributors. [Citace: 9. 3 2025.] <https://smarty-php.github.io/smarty/stable/>.
 20. **THE PHP GROUP.** PHP docs. *PHP*. [Online] PHP contributors. [Citace: 9. 3 2026.] <https://www.php.net/docs.php>.
 21. **NGINX, Inc.** NGINX Documentation. *nginx.org*. [Online] NGINX, Inc. [Citace: 1. 3 2023.] <https://nginx.org/en/docs/>.
 22. **OPENJS FOUNDATION.** jQuery docs. *jQuery*. [Online] OPENJS FOUNDATION. [Citace: 9. 3 2026.] <https://api.jquery.com/>.
 23. **Contributors, Mozilla.** Web docs. *Mozilla*. [Online] Mozilla. [Citace: 6. 3 2026.] <https://developer.mozilla.org/en-US/docs/Web>.
 24. **ÚSTAV GEONIKY AKADEMIE VĚD ČESKÉ REPUBLIKY.** Processing Cookies on Web Pages. [Online] [Citace: 6. 3 2026.] <https://www.ugn.cas.cz/files/Zpracovani-cookies-en.pdf>.
 25. **THE OPEN GROUP.** crontab – schedule periodic background work. *opengroup pubs*. [Online] THE OPEN GROUP. [Citace: 9. 3 2026.] <https://pubs.opengroup.org/onlinepubs/9699919799/utilities/crontab.html>.

A. Seznam příložených souborů

Na přiloženém datovém nosiči se nacházejí následující soubory a složky:

- **MP2026-Havlas-Jakub-P4A-AI_assistent_pro_kvkli.docx** – editovatelná verze dokumentace maturitní práce
- **MP2026-Havlas-Jakub-P4A-AI_assistent_pro_kvkli.pdf** – tisknutelná verze dokumentace maturitní práce
- **Zdrojové kódy** – na github viz QR kód



Obrázek 16 QR na odkaz na github repozitář